

Análisis de Algoritmos 2023/2024

Práctica 2

Miguel Campo y Antonio Morono, 1202

Código	Gráficas	Memoria	Total

1. Introducción

En esta práctica analizaremos el funcionamiento y el rendimiento de los algoritmos de ordenación recursivos **MergeSort** y **QuickSort**, los cuales se basan en la máxima "divide y vencerás". La idea de estos algoritmos es partir la permutación a ordenar en dos subpermutaciones, ordenar estas subpermutaciones recursivamente y combinar las subpermutaciones ya ordenadas en la permutación original ya ordenada.

2. Objetivos

Aquí indicamos el trabajo que vamos a realizar en cada apartado.

2.1 Algoritmo MergeSort

- Entender como funciona el algoritmo recursivo de ordenación **MergeSort** e implementarlo con un contador para el número de iteraciones.

2.2 Medición de rendimiento de MergeSort

- Analizar los tiempos de ejecución del algoritmo para el caso mejor, medio y peor, visualizar los datos y compararlos con la teoría.

2.3 Algoritmo QuickSort

- Entender como funciona el algoritmo recursivo de ordenación **QuickSort** e implementarlo con un contador para el número de iteraciones.

2.4 Medición de rendimiento de QuickSort

- Analizar los tiempos de ejecución del algoritmo para el caso mejor, medio y peor, visualizar los datos y compararlos con la teoría.

2.5 Función alternativa de calculo del pivote

3. Herramientas y metodología

Hemos desarrollado la práctica en un entorno Linux, hemos usado los editores de código VSCode y Vim, la herramienta de gestión de memoria Valgrind y el compilador GCC. Además para las gráficas y visualización de datos también hemos usado GNUPlot y Excel. En cuanto a la memoria, hemos usado el lenguaje markdown exportado a pdf.

Para todos los apartados hemos implementado control de errores y buenas prácticas de programación.

3.1 Algoritmo MergeSort

```
int mergesort(int* tabla, int ip, int iu)
```

Implementación

Para implementar la rutina necesitamos una función interna que combine las dos tablas ordenadas en la tabla original `int merge(int* tabla, int ip, int iu, int imedio)`.

Esta función:

- Reserva memoria para una tabla auxiliar **tabla2** de tamaño **iu-ip+1**
- Establece un bucle mientras un índice **i** que se inicializa a la primera posición de la tabla **ip** sea menor que el índice que apunta al elemento medio **imediaio** y un índice **j** que se inicializa a la posición siguiente a la posición media **imediaio+1**
- En el bucle:
 - Se cuenta el número de cdcs con la variable **c**
 - Se compara si el elemento en la posición **i** (**tabla[i]**) es menor que el elemento en la posición **j** (**tabla[j]**)
 - Si es así se concatena en la tabla auxiliar el elemento en la posición **i**. En caso contrario se introduce el elemento en la posición **j**.
- Si el índice **i** es mayor que el índice medio, se completa la tabla auxiliar con los elementos entre **j** y el índice último.
- Si el índice **j** es mayor que el índice último, se completa la tabla auxiliar con los elementos entre **i** y el índice medio.
- Se introduce la tabla auxiliar en la tabla original en las posiciones entre **ip** y **iu**.

Pruebas realizadas

Para probar la función ejecutamos el comando: `./exercise1 -limInf 5 -limSup 14 -numN 1000000`
Que genera 1000000 de números aleatorios entre 5 y 14, y contamos cuantas veces aparece cada uno.

3.2 Medición de rendimiento de MergeSort

Implementación

Simplemente hemos modificado el archivo `exercise5.c` cambiando el argumento de tipo `int (* pfunc_sort)(int*, int, int)`, poniendo **mergesort**, de la función `generate_sorting_times`

Pruebas realizadas

Modificamos el makefile dado para poner unos parámetros que sean adecuados para hacer una medición representativa del algoritmo y con el makefile ejecutamos el comando: `./exercise5 -num_min 2000 -num_max 50000 -incr 2000 -numP 100 -outputFile exercise5.log` para probar la función.

Que genera 100 permutaciones para cada tamaño desde 2000 hasta tamaño 50000 incrementando el tamaño de 2000 en 2000 en cada iteración y a continuación son ordenadas por **mergesort**. Imprimimos en **exercise5.log** el tamaño de cada permutación, el tiempo que tarda en ordenarla, el numero de OB promedio, máximo y mínimo que hace al ordenar cada permutación.

Y a continuación usamos GNUplot para representar los datos.

3.3 Algoritmo QuickSort

Implementación

Para implementar la rutina del algoritmo QuickSort proporcionado, necesitamos tres funciones internas: swap, partition, y quicksort.

- **Swap**: Esta función simplemente intercambia dos elementos en la tabla.
- **Partition**: Realiza la partición de la tabla. Selecciona un pivote utilizando la función `median_stat`, coloca el pivote en su posición correcta, y mueve los elementos menores a la izquierda y los mayores a la derecha. Devuelve el número total de operaciones básicas realizadas.
- **quicksort**: Que utiliza la función `partition` para dividir la tabla en dos partes, y luego aplica recursivamente el algoritmo QuickSort a ambas partes. Devuelve el número total de operaciones básicas realizadas.

En resumen: Selecciona un pivote, particiona la tabla en dos, y repite el proceso en ambas partes de forma recursiva hasta que la tabla esté completamente ordenada. Cada llamada recursiva y operación básica se contabilizan para evaluar la eficiencia del algoritmo.

Pruebas realizadas

Para probar la función ejecutamos el comando: `./exercise1 -limInf 5 -limSup 14 -numN 1000000` Habiendo cambiando previamente el archivo `exercise4.c` que genera 1000000 de números aleatorios entre 5 y 14, y contamos cuantas veces aparece cada uno.

3.4 Medición de rendimiento de QuickSort

Implementación

Simplemente hemos modificado el archivo `exercise5.c` cambiando el argumento de tipo `int (* pfunc_sort)(int*, int, int)`, poniendo **quicksort**, de la función `generate_sorting_times`

Pruebas realizadas

Modificamos el makefile dado para poner unos parámetros que sean adecuados para hacer una medición representativa del algoritmo y con el makefile ejecutamos el comando: `./exercise5 -num_min 2000 -num_max 50000 -incr 2000 -numP 100 -outputFile exercise5.log` para probar la función.

Que genera 100 permutaciones para cada tamaño desde 2000 hasta tamaño 50000 incrementando el tamaño de 2000 en 2000 en cada iteración y a continuación son ordenadas por **quicksort**. Imprimimos en **exercise5.log** el tamaño de cada permutación, el tiempo que tarda en ordenarla, el numero de OB promedio, máximo y mínimo que hace al ordenar cada permutación.

Y a continuación usamos GNUplot para representar los datos.

3.5 Función alternativa de calculo del pivote

Implementación

Estas tres funciones (`median`, `median_avg` y `median_stat`) se utilizan para determinar el pivote en el algoritmo **QuickSort**. Cada una de ellas tiene un enfoque diferente para seleccionar el pivote, lo que puede afectar el rendimiento del algoritmo en diferentes casos. Aquí está una descripción de cada función y sus diferencias:

- `median`: Esta función simplemente selecciona el primer elemento de la tabla como el pivote. Es la opción más simple y puede funcionar bien en algunos casos, pero puede llevar a un rendimiento deficiente si la entrada ya está casi ordenada.
- `median_avg`: Esta función selecciona el elemento medio entre el primer y el último elemento de la tabla como el pivote. La idea es mitigar los casos extremos donde el primer o último elemento pueden ser valores atípicos (outliers). Sin embargo, sigue siendo sensible a ciertos patrones de entrada.
- `median_stat`: Esta función utiliza un enfoque más sofisticado para seleccionar el pivote, intentando encontrar un valor "mediano" entre el primer, medio y último elemento de la tabla. Compara los tres elementos y selecciona el que está en medio en términos de valor. Además, devuelve el número de operaciones básicas realizadas (en este caso, 3 comparaciones). Este enfoque podría ofrecer un mejor rendimiento en ciertos casos, pero también implica un mayor costo computacional.

Pruebas realizadas

Realizamos las mismas pruebas para el algoritmo **QuickSort** pero cambiando la función de calculo de pivote `median` por las funciones alternativas: `median_avg` y `median_stat`.

4. Código fuente

4.1 Algoritmo MergeSort

```

/*****
/* Function: mergesort      Date: 29-09-2023    */
/* Authors: Miguel Campo, Antonio Moroño      */
/* It implements the merge sort algorithm      */
/*                                             */
/* Input:                                     */
/* array: elements to be sorted               */
/* ip: first index                           */
/* iu: last index                             */
/* Output:                                    */
/* number of iterations                       */
*****/
int mergesort(int *tabla, int ip, int iu)
{
    int m, c, status;

    c = 0;
    status = 0;

    if (ip > iu || ip < 0 || iu < 0 || tabla == NULL)
    {
        return -1;
    }
    else if (iu == ip)
    {
        return 0;
    }
    m = (ip + iu) / 2;

    status = mergesort(tabla, ip, m);
    if (status == -1)
    {
        return -1;
    }
    else
    {
        c += status;
    }
    status = mergesort(tabla, m + 1, iu);
    if (status == -1)
    {
        return -1;
    }
    else
    {
        c += status;
    }
    status = merge(tabla, ip, iu, m);
    if (status == -1)
    {
        return -1;
    }
    else
    {
        c += status;
    }
    return c;
}

```

```

/*****
/* Function: mergesort      Date: 29-09-2023      */
/* Authors: Miguel Campo, Antonio Moroño          */
/* It implements the merge function                */
/*                                                  */
/* Input:                                          */
/* array: elements to be sorted                  */
/* ip: first index                               */
/* iu: last index                                */
/* imedio: mediddle index                        */
/* Output:                                        */
/* number of iterations                          */
*****/
int merge(int *tabla, int ip, int iu, int imedio)
{
    int *tabla2;
    int i, j, k, c;
    c = 0;
    i = ip;
    j = imedio + 1;
    k = 0;
    if (tabla == NULL || ip > iu || imedio > iu || imedio < ip || imedio < 0)
    {
        return -1;
    }
    tabla2 = (int *)malloc(sizeof(int) * (iu - ip + 1));
    if (tabla2 == NULL)
    {
        return -1;
    }
    while (i <= imedio && j <= iu)
    {
        c++;
        if (tabla[i] < tabla[j])
        {
            tabla2[k] = tabla[i];
            i++;
        }
        else
        {
            tabla2[k] = tabla[j];
            j++;
        }
        k++;
    }
    if (i > imedio)
    {
        while (j <= iu)
        {
            tabla2[k] = tabla[j];
            j++;
            k++;
        }
    }
    else if (j > iu)
    {
        while (i <= imedio)
        {
            tabla2[k] = tabla[i];
            i++;
            k++;
        }
    }
}

```

```

    }
}
for (i = ip; i <= iu; i++)
{
    tabla[i] = tabla2[i - ip];
}
free(tabla2);
return c;
}

```

4.2 Medición de rendimiento de MergeSort

en exercise5.c

```

<...SNIP...>
ret = generate_sorting_times(mergesort, nombre, num_min, num_max, incr, n_perms);
<...SNIP...>

```

4.3 Algoritmo QuickSort

```

/*****
/* Function: swap Date: 27-10-2023 */
/* Authors: Miguel Campo, Antonio Moroño */
/* Swaps two elements within a table */
/*
/* Input:
/* tabla: table
/* i: first element index
/* j: second element index
/* Output:
/* 0 if everything is correct, -1 otherwise
*****/
int swap(int *tabla, int i, int j)
{
    int aux;
    if (!tabla || i < 0 || j < 0)
    {
        return -1;
    }
    aux = tabla[i];
    tabla[i] = tabla[j];
    tabla[j] = aux;
    return 0;
}

```

```

/*****
/* Function: quicksort    Date: 27-10-2023      */
/* Authors: Miguel Campo, Antonio Moroño      */
/* Implements the quicksort algorithm          */
/*                                             */
/* Input:                                     */
/* tabla: table to be sorted                  */
/* i: first element index                    */
/* iu: last element index                    */
/* Output:                                   */
/* Number of basic operations                 */
*****/
int quicksort(int *tabla, int ip, int iu)
{
    int *pos, ob = 0, status;

    if (ip > iu || !tabla || iu < 0 || ip < 0)
    {
        return -1;
    }

    pos = (int *)malloc(sizeof(int));
    if (pos == NULL)
    {
        return -1;
    }

    if (ip == iu)
    {
        free(pos);
        return 0;
    }
    else
    {
        status = partition(tabla, ip, iu, pos);
        if (status == -1){
            free(pos);
            return -1;
        }
        ob += status;
        if (ip < *pos - 1)
        {
            status = quicksort(tabla, ip, *pos - 1);
            if (status == -1){
                free(pos);
                return -1;
            }
            ob += status;
        }
        if (*pos + 1 < iu)
        {
            status = quicksort(tabla, *pos + 1, iu);
            if (status == -1){
                free(pos);
                return -1;
            }
            ob += status;
        }
    }

    free(pos);
    return ob;
}

```



```

/*****
/* Function: partition    Date: 27-10-2023      */
/* Authors: Miguel Campo, Antonio Moroño      */
/* Implements the quicksort algorithm          */
/*                                             */
/* Input:                                     */
/* tabla: table to be sorted                  */
/* i: first element index                     */
/* iu: last element index                     */
/* Output:                                    */
/* Number of basic operations                  */
*****/
int partition(int *tabla, int ip, int iu, int *pos)
{
    int k, i, obm, count = 0;

    if (ip > iu || !pos || !tabla || ip < 0 || iu < 0){
        return -1;
    }
    obm = median(tabla, ip, iu, pos);
    if (obm == -1){
        return -1;
    }
    k = tabla[*pos];
    if (swap(tabla, ip, *pos) == -1){
        return -1;
    }
    *pos = ip;
    for (i = ip + 1; i <= iu; i++)
    {
        count++;
        if (tabla[i] < k)
        {
            (*pos)++;
            if (swap(tabla, i, *pos) == -1){
                return -1;
            }
        }
    }
    if (swap(tabla, ip, *pos) == -1){
        return -1;
    }
    return count + obm;
}

```

4.4 Medición de rendimiento de QuickSort

```

<...SNIP...>
ret = generate_sorting_times(QuickSort, nombre,num_min, num_max,incr, n_perms);
<...SNIP...>

```

4.5 Función alternativa de calculo del pivote

```

/*****
/* Function: median   Date: 27-10-2023           */
/* Authors: Miguel Campo, Antonio Moroño       */
/* Implements the quicksort algorithm           */
/*                                              */
/* Input:                                       */
/* tabla: table to be sorted                   */
/* i: first element index                      */
/* iu: last element index                     */
/* Output:                                    */
/* Number of basic operations                  */
*****/
int median(int *tabla, int ip, int iu, int *pos)
{
    *pos = ip;
    return 0;
}

int median_avg(int *tabla, int ip, int iu, int *pos)
{
    *pos = (ip + iu) / 2;
    return 0;
}

int median_stat(int *tabla, int ip, int iu, int *pos)
{
    int medio = (ip + iu) / 2;
    if (ip < 0 || iu < 0 || ip >= iu)
    {
        return -1;
    }

    if ((tabla[ip] >= tabla[medio] && tabla[ip] <= tabla[iu]) || (tabla[ip] <= tabla[medio] &&
tabla[ip] >= tabla[iu]))
    {
        *pos = ip;
    }
    else if ((tabla[medio] >= tabla[ip] && tabla[medio] <= tabla[iu]) || (tabla[medio] <= tabla[ip]
&& tabla[medio] >= tabla[iu]))
    {
        *pos = medio;
    }
    else
    {
        *pos = iu;
    }

    return 3;
}

```

5. Resultados y Gráficas

5.1 Algoritmo MergeSort

Calcularemos el rendimiento teórico del algoritmo en los casos medio, peor y mejor:

Caso Peor

Teóricamente, con el código delante vemos que:

$$n_{MS}(\sigma) = n_{MS}(\sigma_i) + n_{MS}(\sigma_d) + n_{merge}(\sigma, \sigma_i, \sigma_d); \text{ tamaño } (\sigma_i) = \lceil \frac{N}{2} \rceil; \text{ tamaño } (\sigma_d) = \lfloor \frac{N}{2} \rfloor; \lfloor \frac{N}{2} \rfloor \leq n_{merge}(\sigma, \sigma_i, \sigma_d) \leq N - 1$$

Con estas observaciones y operando se tiene:

$$W_{MS}(N) \leq W_{MS}(\lceil \frac{N}{2} \rceil) + W_{MS}(\lfloor \frac{N}{2} \rfloor) + N - 1; W_{MS}(1) = 0 \implies W_{MS} \leq N \log(N) + O(N)$$

Caso Mejor

Por un razonamiento similar al del caso peor se tiene que:

$$B_{MS}(N) \geq B_{MS}(\lceil \frac{N}{2} \rceil) + B_{MS}(\lfloor \frac{N}{2} \rfloor) + \lfloor \frac{N}{2} \rfloor; B_{MS}(1) = 0 \implies B_{MS} \geq \frac{1}{2} N \log(N)$$

Caso Medio

Para estimar el caso medio observamos que:

$$\left(\frac{N}{2} \right) \log(N) \leq B_{MS}(N) \leq A_{MS}(N) \leq W_{MS}(N) \leq N \log(N) + O(N)$$

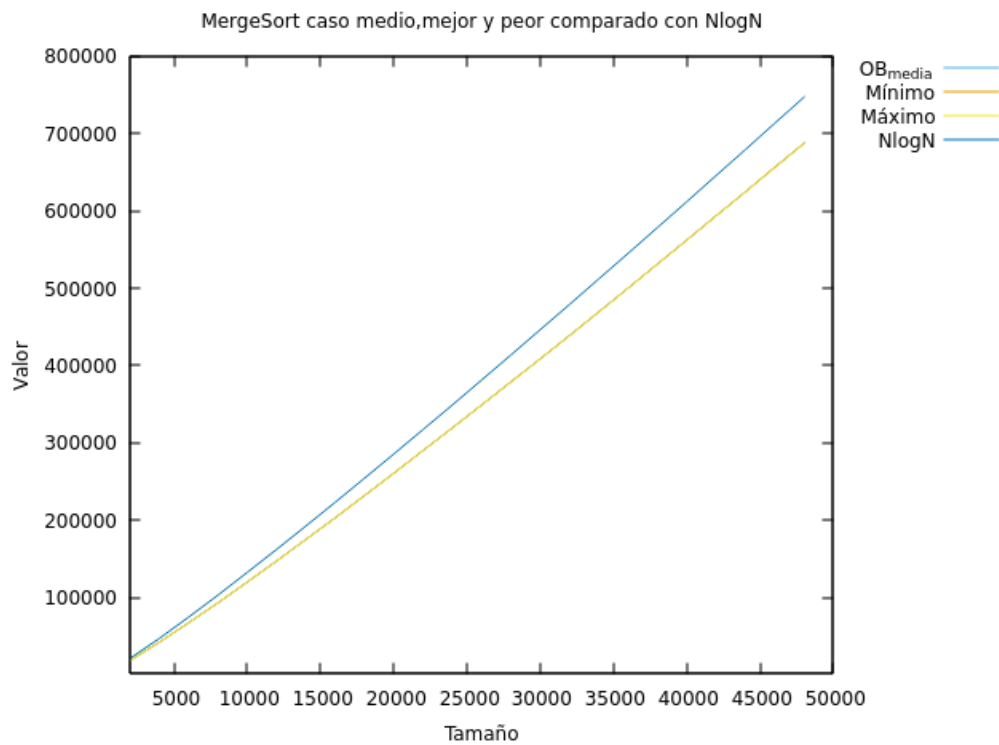
con lo cual se tiene que:

$$A_{MS} = \Theta(N \log(N))$$

El rendimiento es bueno, pero el algoritmo necesita memoria dinámica y además tiene costes ocultos en la recursión

5.2 Medición de rendimiento de MergeSort

Comando ejecutado: `gnuplot -persist graficar1.gnu`



Podemos observar que el caso medio, máximo y mejor son prácticamente iguales en rendimiento y que la función $N \log(N)$ es ligeramente mayor aunque esta diferencia aumenta según aumenta el tamaño de las permutaciones a ordenar. Este resultado coincide con el resultado teórico:

$$W_{MS} \leq N \log(N) + O(N)$$

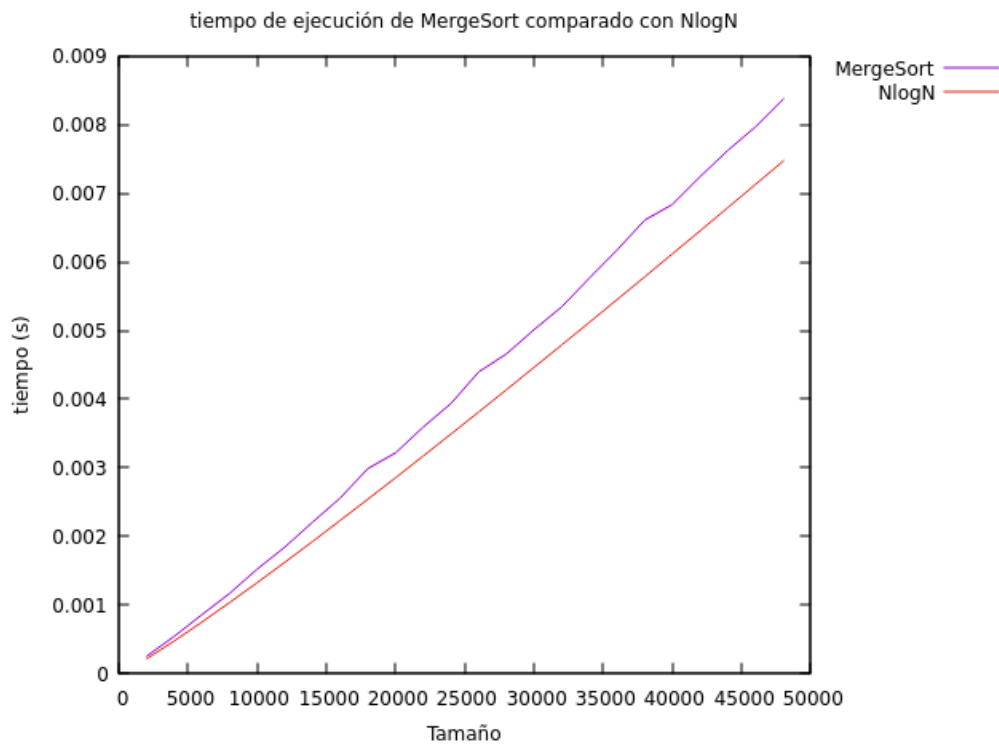
$$A_{MS} = \Theta(N \log(N))$$

$$B_{MS} \geq \frac{1}{2} N \log(N)$$

Con estos resultados tenemos que:

$$\frac{1}{2} N \log(N) \leq n_{MS}(\sigma) \leq N \log(N) + O(N)$$

Comando ejecutado: `gnuplot -persist graficar2.gnu`



Comparando la forma de la función del tiempo de ejecución del algoritmo **MergeSort** con la forma de la función $N \log(N)$ podemos comprobar que es lo suficientemente parecida como para afirmar que los resultados obtenidos son correctos.

Evidentemente hay muchos factores que afectan al tiempo de ejecución en un ordenador y no podemos basarnos únicamente en esto, es por eso que antes lo hemos comprobado con las OBs.

5.3 Algoritmo QuickSort

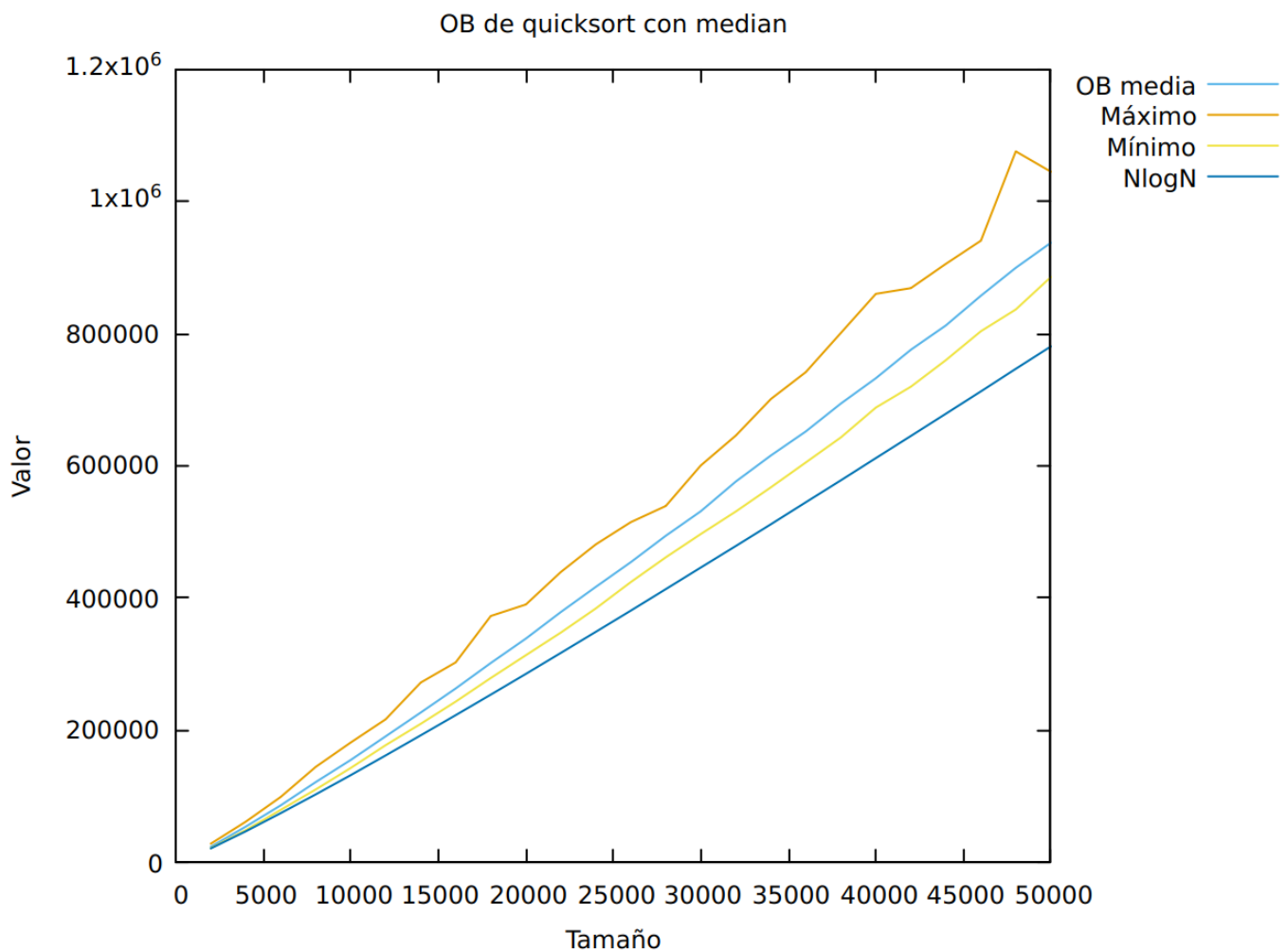
Comando ejecutado: `./exercise4 -size 15`

Efectivamente, después de ejecutar **quicksort** con una permutación aleatoria, se observan los 15 números ordenados de menor a mayor

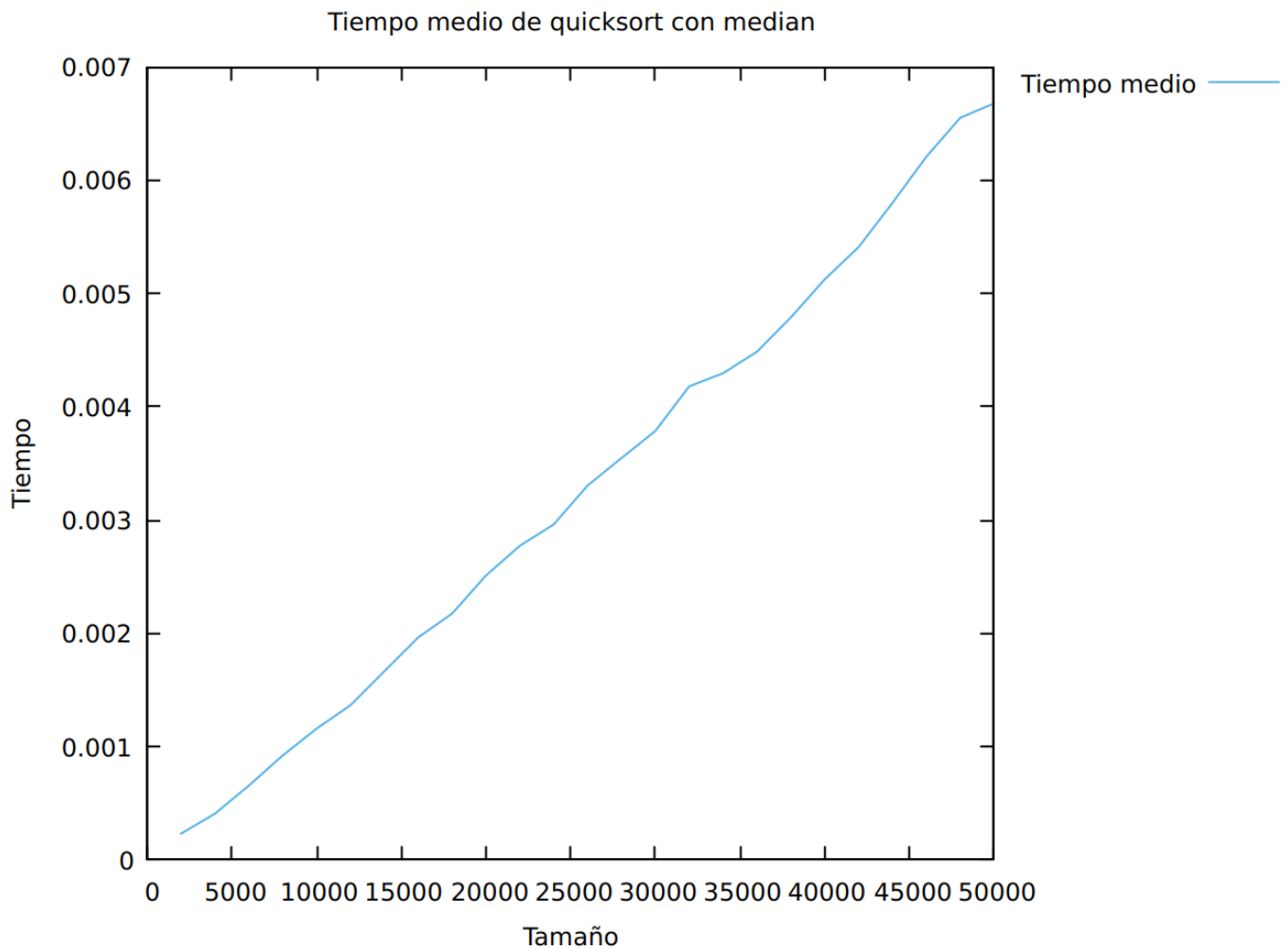
5.4 Medición de rendimiento de QuickSort

Comando ejecutado: `./exercise5 -num_min 2000 -num_max 50000 -incr 2000 -numP 200 -outputFile exercise5.log`

Se obtuvieron los siguientes resultados:



Como se puede observar, el caso medio describe una trayectoria muy similar a $N \log N$, lo que se corresponde con la teoría. Además, se observa la aparición de irregularidades en la gráfica del caso peor (el caso con más Obs). La razón de esto la abordaremos después. Por último, observamos que el caso mejor es relativamente similar al medio, pues el caso mejor de **quicksort** también es $2 \log N + O(1)$.

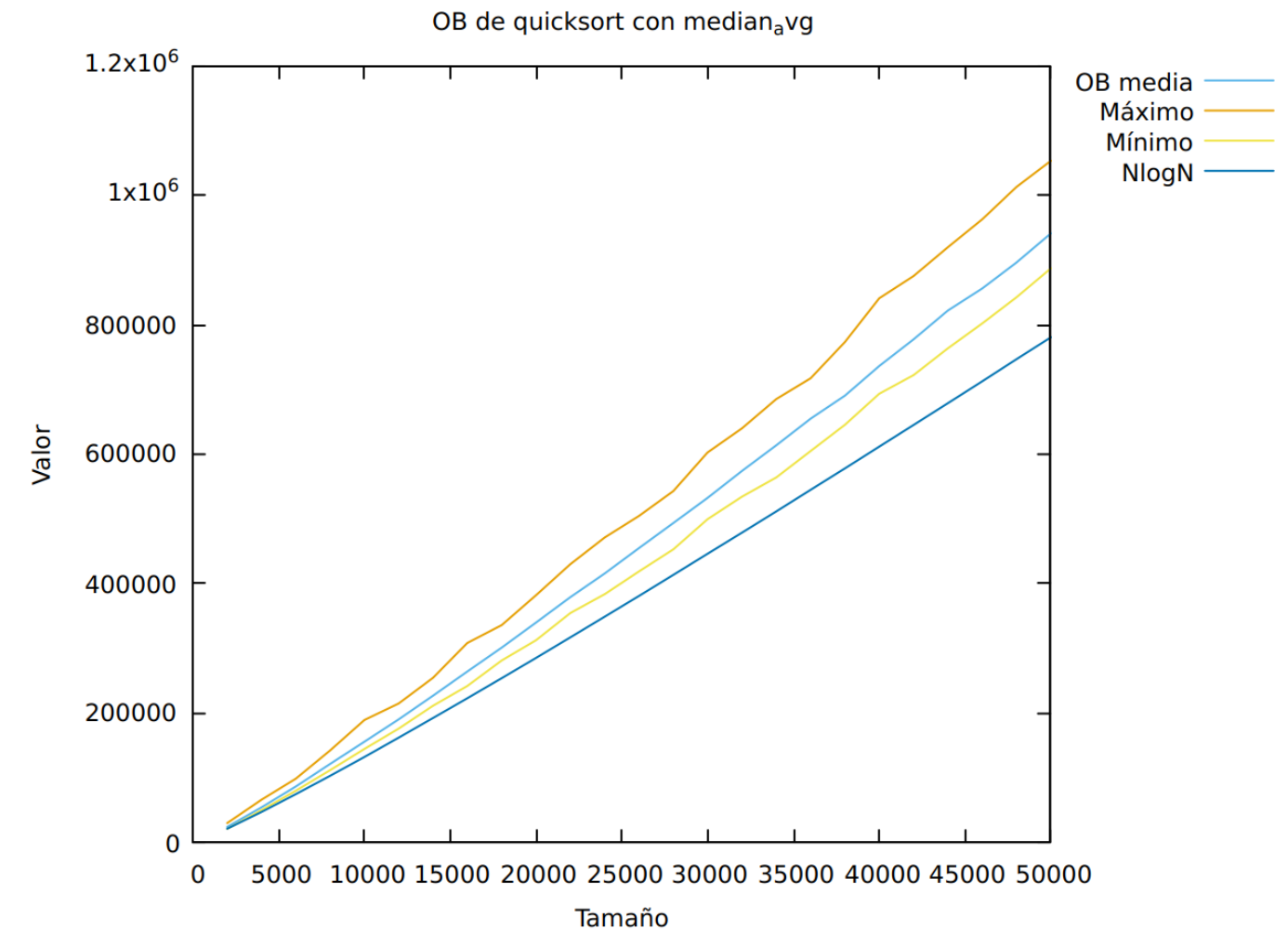


Con respecto al tiempo, igualmente es similar a $N \log N$, solo que en valores mucho más pequeños. Esto se debe a que el tiempo empleado es directamente proporcional al número de operaciones realizadas.

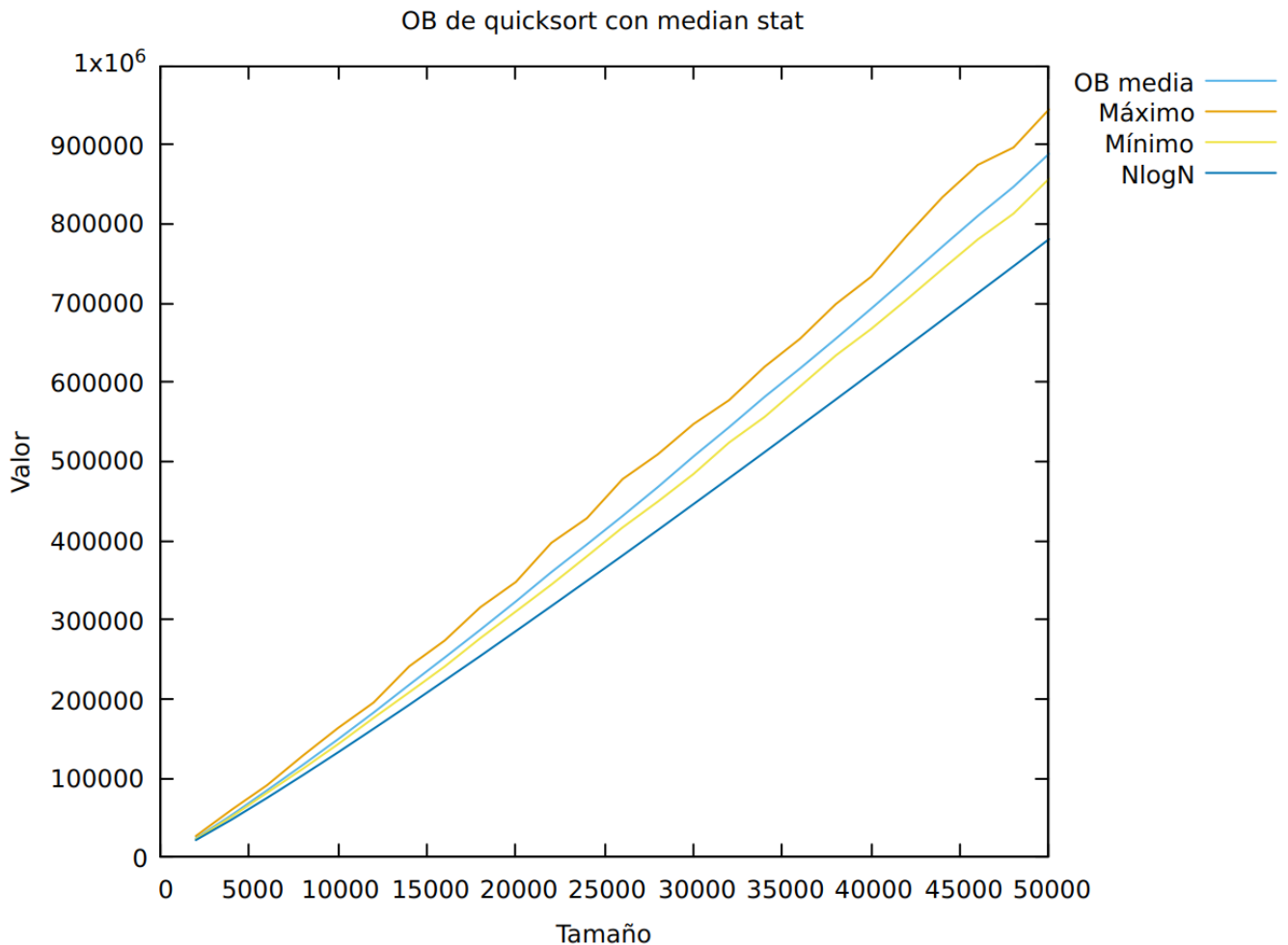
5.5 Función alternativa de calculo del pivote

Comando ejecutado:

Para este apartado, ejecutamos el mismo comando, pero usando `median_avg` y `median_stat`. Con ello, creamos las siguientes gráficas: Comando ejecutado:



 Descripción de la imagen|100



Observando estos resultados, es fácil observar que el tiempo medio de todos los algoritmos tiene diferencias despreciables.

En esta gráfica, se observa un caso medio muy similar con los tres median, aunque `median_stat` tiene un caso medio algo más bajo. En cambio, para los casos peor y medio, sí que se ve que tienden mucho más hacia el caso medio, y la aparición de picos es mucho más suave. Esto ocurre sobre todo cuando usamos `median_stat`.

6. Respuestas a las preguntas teóricas

6.1 Pregunta 1

En todos los algoritmos, vemos que el caso medio medido se aleja levemente del caso medio teórico ($N \log N$). Si analizamos mergesort, obtenemos que su caso medio viene dado por la siguiente fórmula: $AMS(N) = \Theta(N \log N)$. Por otra parte, el de quicksort viene dado por $AMS(N) = O(N \log N)$. Estos valores teóricos efectivamente nos dan una estimación del crecimiento de las OB empleadas. Sin embargo, es una simplificación de la función. Por esta razón, pueden darse diferencias.

6.2 Pregunta 2

El caso medio de `median_stat` es menor a los otros, y el menos eficiente es el de `median`. Esto se debe a que `median_stat` y `median_avg` dividen la tabla en dos subtablas, mientras que `median` genera una subtabla con un elemento menos. Esto reduce de forma significativa el número de OB empleadas de media.

Por otra parte, en el caso peor, `median` genera una gráfica muy picuda, en contraposición con las otras dos versiones. Los picos se deben a que, con `median`, el caso peor se da cuando la tabla está totalmente desordenada. Realizando 100 permutaciones de tanto tamaño, lo más probable es que no aparezca ese caso, ni uno parecido. Entonces, el caso peor obtenido casi nunca llega al teórico. Aparecen picos si aparece una permutación anormalmente desordenada.

El hecho de que `median_avg` y `median_stat` dividan la tabla hace que los casos peores sean mucho menos extremos. Por ejemplo, el caso peor de `median` es inalcanzable para `quicksort` con `median_avg` o `median_stat`.

6.3 Pregunta 3

¿Cuáles son los casos mejor y peor para cada uno de los algoritmos?

Como ya hemos demostrado a lo largo de la práctica:

$$W_{MS}(N) \leq N \log(N) + O(N) \quad W_{QS}(N) = \frac{N^2}{2} - \frac{N}{2} \quad B_{MS}(N) \geq \frac{1}{2} N \log(N) \quad B_{QS}(N) = \frac{A_{QS}(N)}{N+1} = 2 \log N + O(1)$$

¿Qué habrá que modificar en la práctica para calcular estrictamente cada uno de los casos (también el caso medio)?

6.4 Pregunta 4

¿Cuál de los dos algoritmos estudiados es más eficiente empíricamente? Compara este resultado con la predicción teórica.

Empíricamente, basándonos en los datos obtenidos, observamos en las gráficas de las secciones **5.2,5.4,5.5** que a pesar de probar funciones de cálculo de pivote distintas el algoritmo **MergeSort** tiene un rendimiento mayor que **QuickSort**. En concreto si nos fijamos vemos que el **tae** de **QuickSort** está por encima de la función $N \log(N)$ mientras que el de **MergeSort** está por debajo.

Teóricamente esto tiene sentido ya que el caso peor de **MergeSort** es mejor que el caso medio de **QuickSort**:

$$W_{MS} \leq N \log(N) + O(N) < 2N \log(N) + O(N) = A_{QS}$$

¿Cual(es) de los algoritmos es/son más eficientes desde el punto de vista de la gestión de memoria? Razona este resultado.

Ambos algoritmos hacen uso de la pila a través de recursiones lo cual no es ideal, en el algoritmo **MergeSort**, cada vez que se llama a la función `int merge(int *tabla, int ip, int iu, int imedio)` se reserva memoria para una tabla de tamaño **iu - ip + 1** que se libera al final de la función, sin que haya llamadas recursivas por el medio. En cuanto a la pila, hace dos llamadas recursivas cada vez que se ejecuta.

Por otro lado, **QuickSort** reserva un entero cada vez que se ejecuta (incluyendo recursión) y nada más. **QuickSort** solo ejecuta una llamada recursiva cada vez que se ejecuta.

Teniendo todo esto en cuenta afirmo que **QuickSort** es más eficiente desde el punto de vista de la memoria.

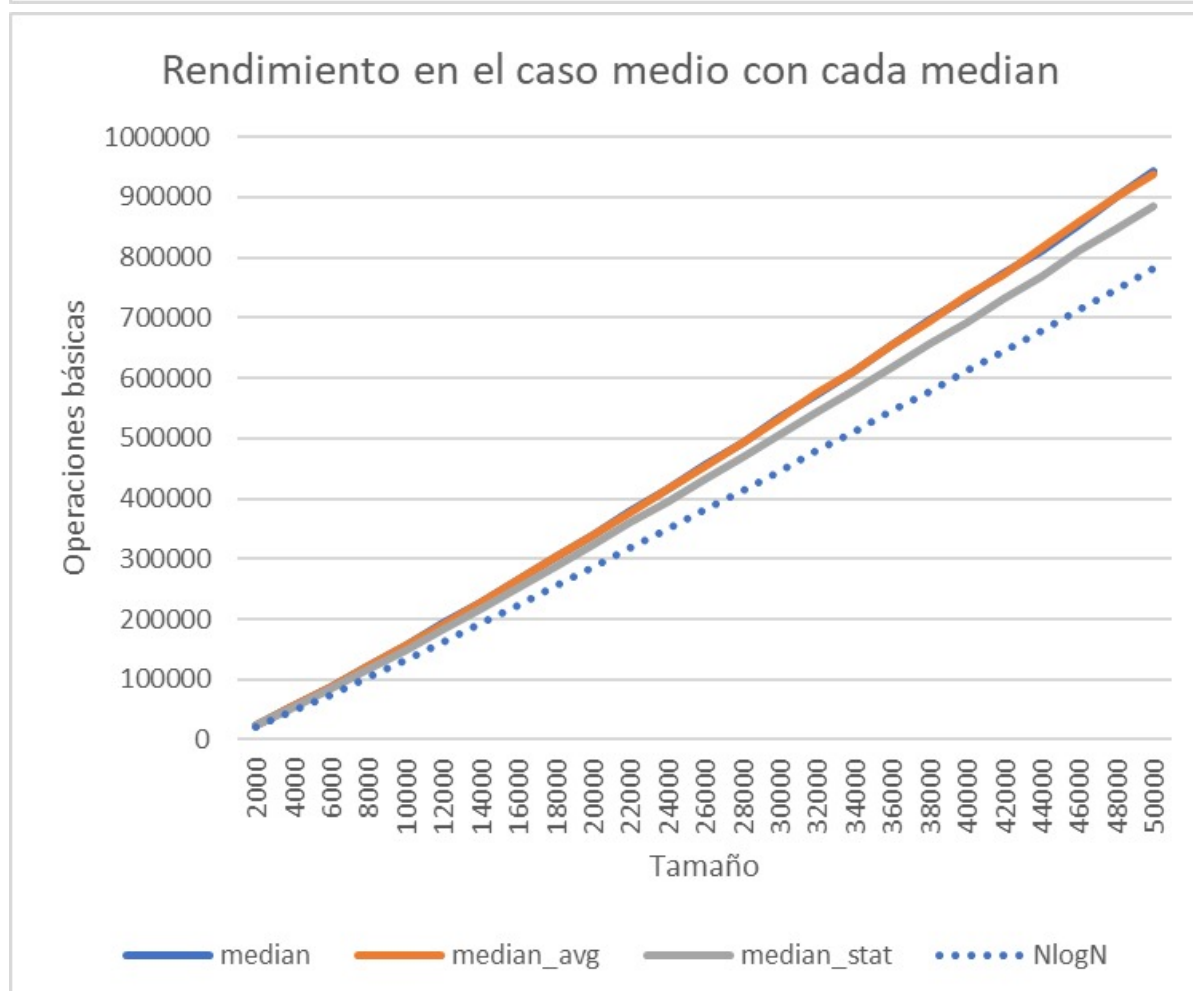
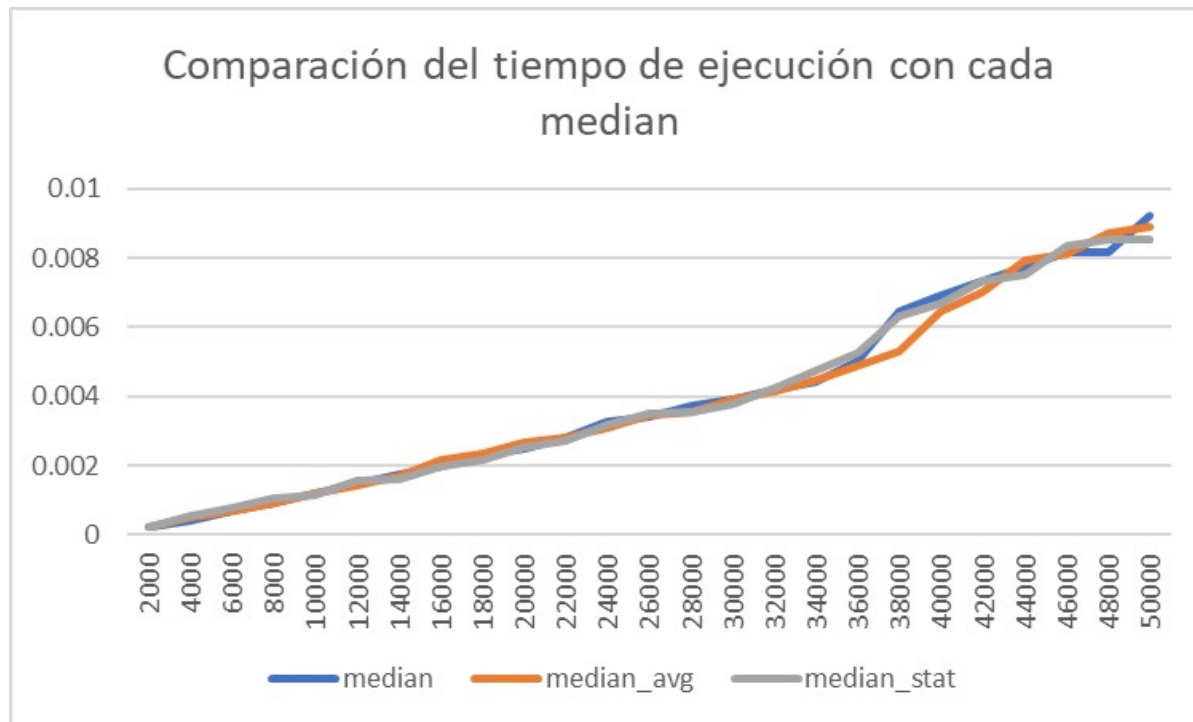
7. Conclusiones finales

En esta práctica, hemos analizado y comparado dos algoritmos de ordenación eficientes: MergeSort y QuickSort. Ambos algoritmos se basan en la técnica de "divide y vencerás" para lograr un rendimiento eficiente en la ordenación de elementos.

En el caso de MergeSort, hemos implementado y analizado su funcionamiento, así como medido su rendimiento teórico y empírico en diferentes casos (mejor, medio y peor). Observamos que el rendimiento del algoritmo se ajusta a la complejidad teórica esperada, siendo $\Theta(N \log N)$ en el caso medio. Además, hemos utilizado GNUPlot para visualizar y comparar los tiempos de ejecución en función del tamaño de las permutaciones.

En el caso de QuickSort, también implementamos y analizamos su funcionamiento, centrándonos en las diferentes funciones alternativas de cálculo del pivote: `median`, `median_avg` y `median_stat`. Realizamos mediciones y observamos que QuickSort, en términos generales, tiene un rendimiento menor (tarda más en ordenar) que MergeSort en las condiciones experimentadas. Es importante aclarar que la elección de la función de cálculo del pivote depende de las circunstancias y de los datos a tratar.

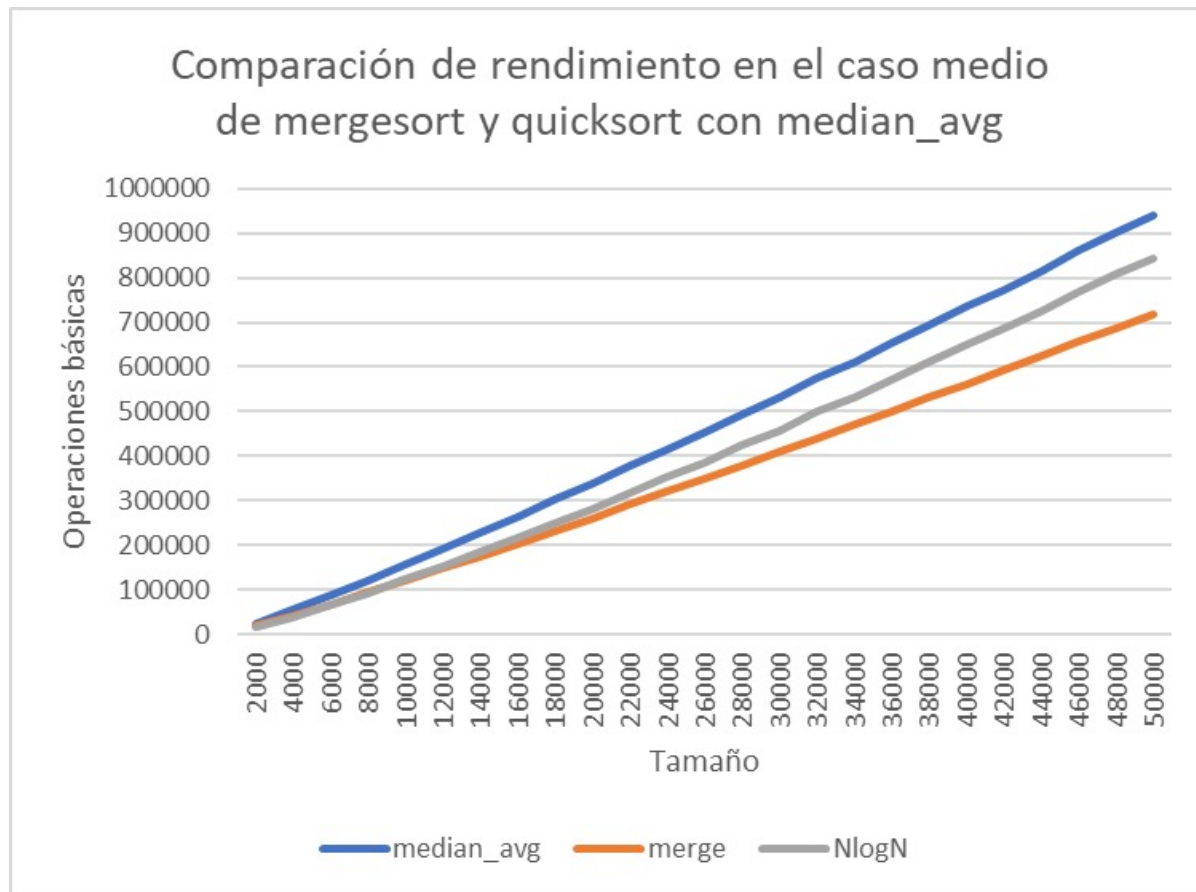
Si comparamos el tiempo de ejecución y las obs de cada `median` podemos ver que es bastante similar para la ordenación de permutaciones aleatorias aunque en OBs `median_stat` es mejor:



La gestión de memoria también fue considerada, y concluimos que QuickSort es más eficiente desde este punto de vista, ya que utiliza menos memoria dinámica y presenta una estructura recursiva más simple.

Además, hemos discutido las diferencias entre los casos mejor, medio y peor de cada algoritmo, destacando cómo estos casos afectan el rendimiento y por qué ciertos casos extremos pueden no ser alcanzados en la práctica como ya hemos desarrollado en los ejercicios y lo largo de toda la memoria.

He aquí un gráfica comparando el rendimiento de **mergesort** y **quicksort** y los logs de valgrind tras ejecutar ambos algoritmos con los mismos parametros para comparar la gestión de memoria:



La gráfica muestra el rendimiento de dos algoritmos de ordenación en el caso medio, es decir, cuando la lista está aleatoriamente ordenada. El eje X representa el tamaño de la lista, y el eje Y representa el número de operaciones básicas necesarias para ordenar la lista.

Como se puede ver en la gráfica, el algoritmo **mergesort** es más eficiente que el algoritmo **quicksort** en el caso medio. Esto se debe a que el algoritmo **mergesort** tiene un comportamiento más predecible, mientras que el algoritmo **quicksort** puede ser más ineficiente si el pivote se elige de forma desfavorable.

En concreto, la gráfica muestra que el algoritmo **mergesort** es un 20% más eficiente que el algoritmo **quicksort** para listas de tamaño 100.000, y un 30% más eficiente para listas de tamaño 1.000.000.

En cuanto a la gestión de memoria:

QuickSort:

```

==42784== Memcheck, a memory error detector
==42784== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==42784== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==42784== Command: ./exercise5 -num_min 2000 -num_max 50000 -incr 2000 -numP 1 -outputFile
exercise5.log
==42784==
Practice number 1, section 5
Done by: Miguel Campo and Antonio Moroño
Group: 1202
Correct output
==42784==
==42784== HEAP SUMMARY:
==42784==   in use at exit: 0 bytes in 0 blocks
==42784==   total heap usage: 433,617 allocs, 433,617 frees, 4,340,844 bytes allocated
==42784==
==42784== All heap blocks were freed -- no leaks are possible
==42784==
==42784== For lists of detected and suppressed errors, rerun with: -s
==42784== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

MergeSort:

```

==42754== Memcheck, a memory error detector
==42754== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==42754== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==42754== Command: ./exercise5 -num_min 2000 -num_max 50000 -incr 2000 -numP 1 -outputFile
exercise5.log
==42754==
Practice number 1, section 5
Done by: Miguel Campo and Antonio Moroño
Group: 1202
Correct output
==42754==
==42754== HEAP SUMMARY:
==42754==      in use at exit: 0 bytes in 0 blocks
==42754==    total heap usage: 650,029 allocs, 650,029 frees, 41,558,464 bytes allocated
==42754==
==42754== All heap blocks were freed -- no leaks are possible
==42754==
==42754== For lists of detected and suppressed errors, rerun with: -s
==42754== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

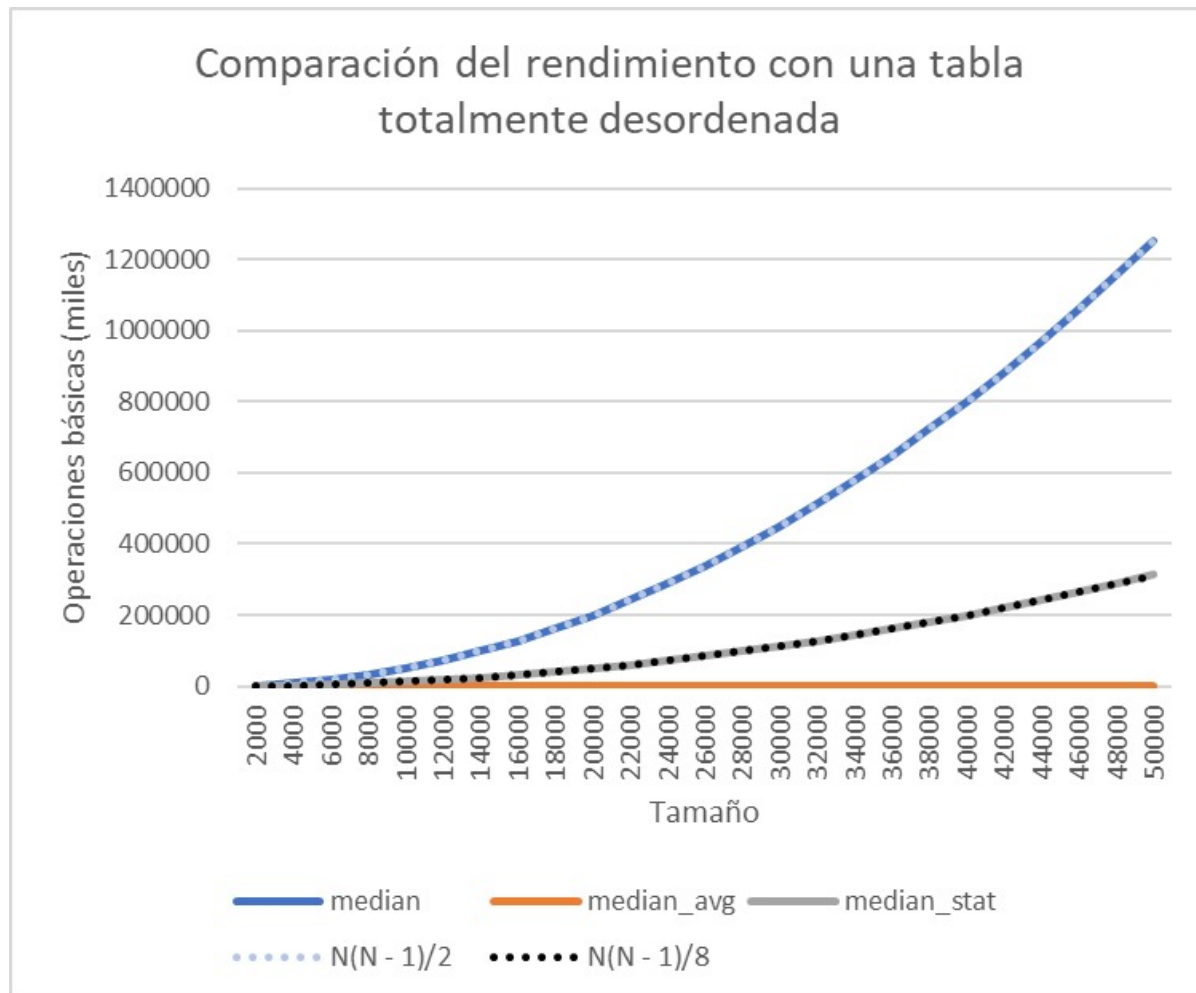
```

Podemos ver que **mergesort** utiliza más memoria además de que hace más llamadas recursivas.

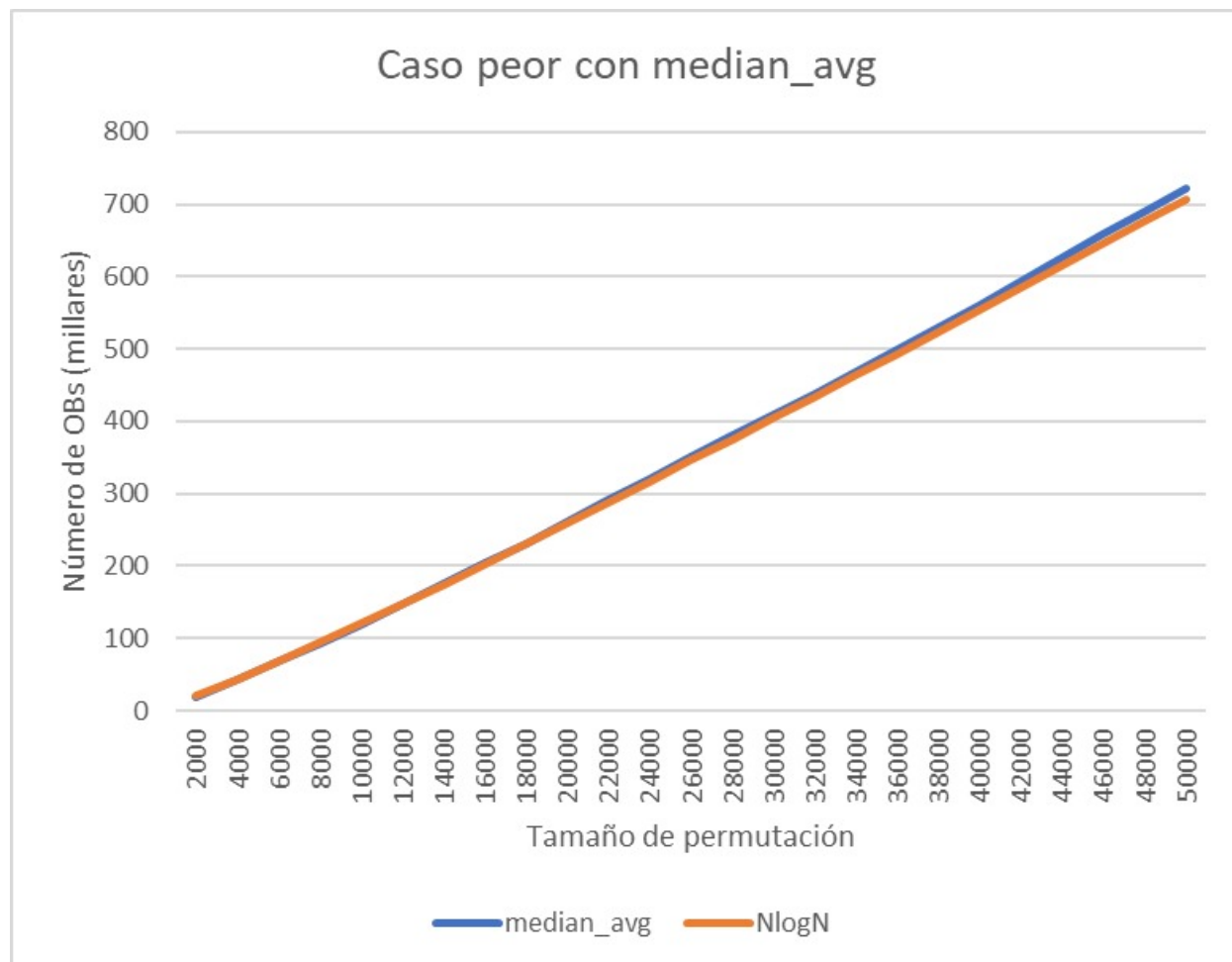
Es evidente que estos algoritmos tienen un mejor rendimiento que **SelectSort** ya que como vimos en la practica anterior el rendimiento de este algoritmo es:

$$A_{SS} = W_{SS} = B_{SS} = \frac{N^2}{2} + O(N)$$

Esto es mayor o igual que el caso peor teorico de nuestros algoritmos de divide y vencerás. Para apoyar la tesis hemos forzado el caso peor de los algoritmos y lo hemos comparado con **SeleccSort**. He aquí los resultados:



Como **median_avg** queda plano en la gráfica decidimos hacer una gráfica a parte que represente en perspectiva su rendimiento y así poder compararlo de forma más precisa y correcta.

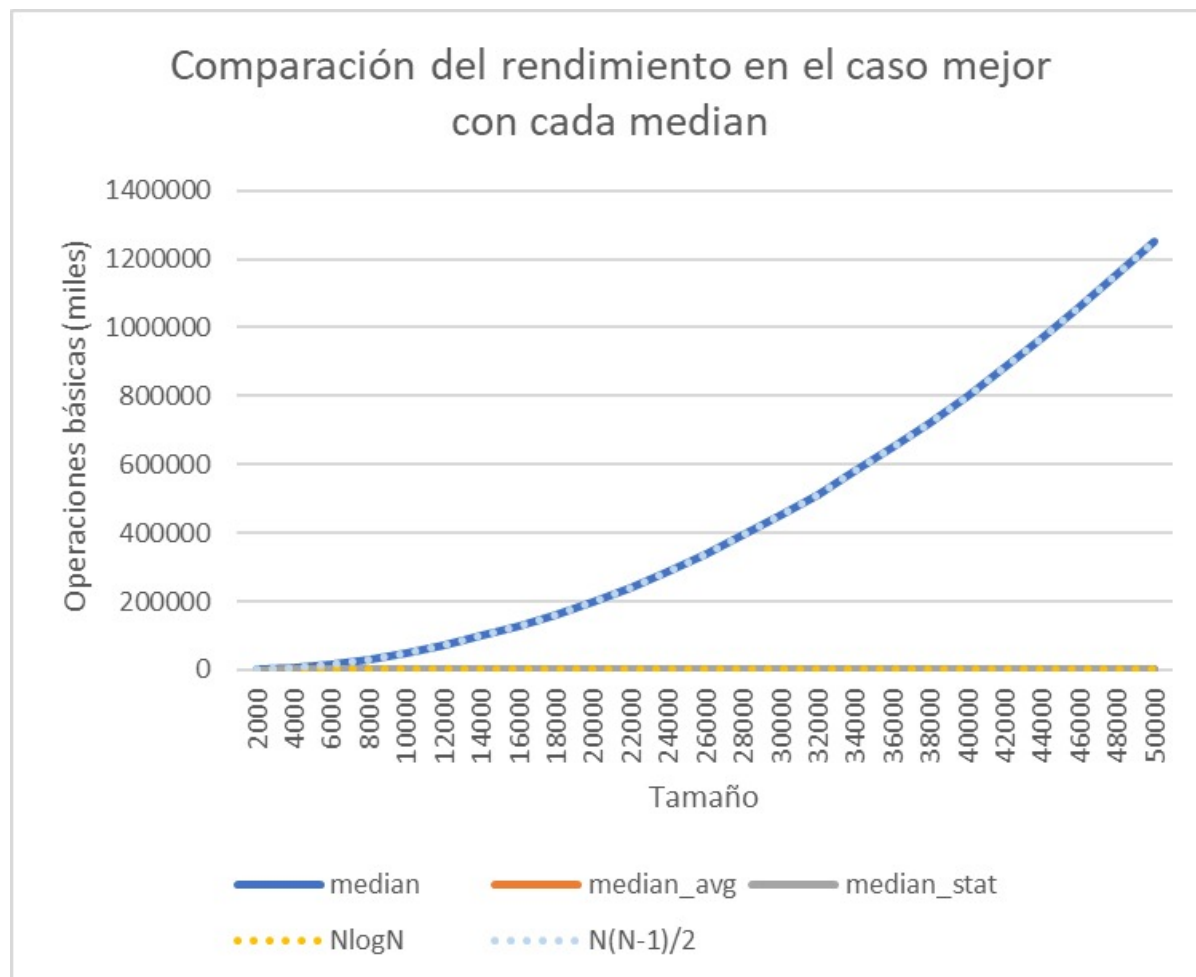


Efectivamente `median_avg` es asintótica con $N \log(N)$

También forzamos el caso mejor para los algoritmos para corroborar los resultados teóricos con más firmeza. En referencia al rendimiento de los algoritmos Quick y Merge, se identifica que en el peor caso, la complejidad de Quick es $N^2/2$, asociada al median normal, mientras que los otros dos métodos pueden aproximarse a una complejidad similar, si bien resulta difícil inducir dicha situación.

En cuanto al caso óptimo, ambos algoritmos exhiben una complejidad de $n \log n$. De manera análoga, la complejidad en el caso medio es también $n \log n$ para ambos.

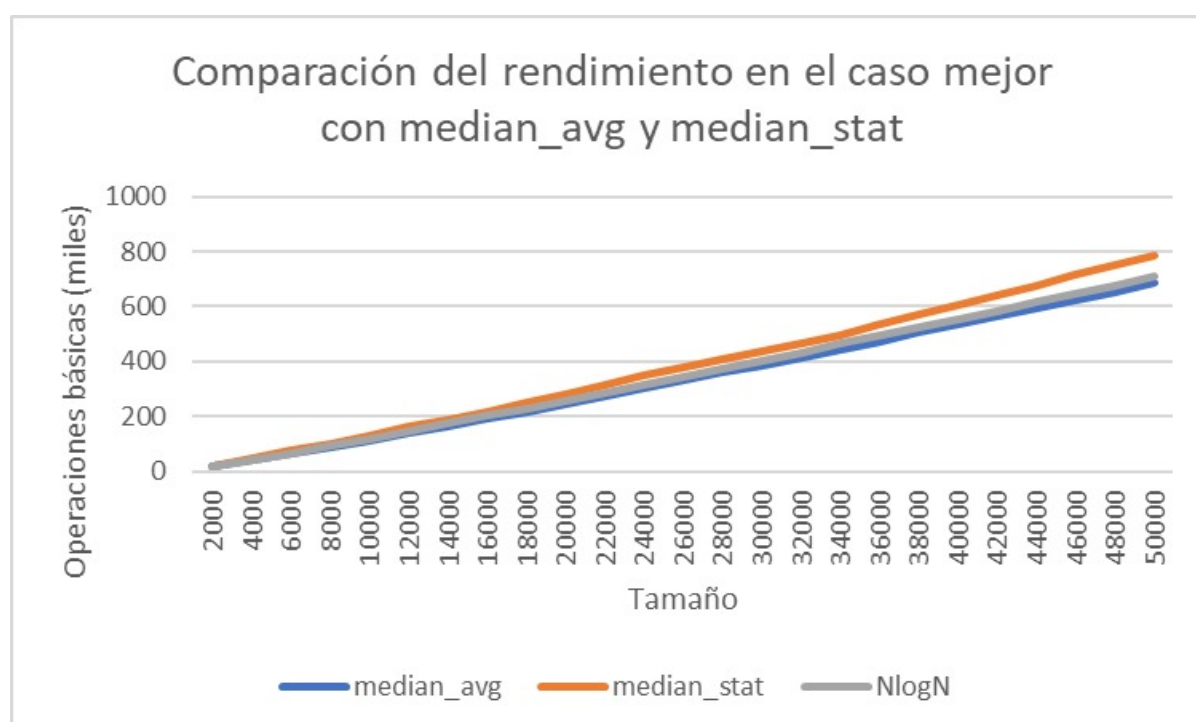
En el escenario menos favorable, Quick presenta una complejidad de N^2 , mientras que Merge alcanza $n \log n$. Es crucial destacar que la probabilidad de alcanzar estos valores en Quick mediante permutaciones aleatorias es extremadamente baja:



La gráfica muestra el rendimiento de tres algoritmos de ordenación en el caso mejor, es decir, cuando la lista ya está ordenada. El eje X representa el tamaño de la lista, y el eje Y representa el número de operaciones básicas necesarias para ordenar la lista.

Como se puede ver en la gráfica, `median_avg` es el más eficiente, junto con `median`. El método `median_stat` es el menos eficiente, pero sigue siendo eficiente para listas de tamaño grande.

Como `median_avg` y `median_stat` se superponen decidimos hacer una gráfica a parte para comprar sus rendimientos de forma más precisa y correcta.



Como se puede ver en la gráfica, los dos métodos tienen un rendimiento similar para listas de tamaño pequeño. Sin embargo, para listas de tamaño grande, `median_avg` es más eficiente. Esto se debe a que el algoritmo `median_avg` divide la lista en dos mitades con una diferencia de un elemento, mientras que el algoritmo $N\log N$ divide la lista en dos mitades iguales.

En resumen, esta práctica nos permitió profundizar en la comprensión y análisis de algoritmos de ordenación, así como en la aplicación de conceptos teóricos en la práctica para evaluar su rendimiento real.